

MCP Apps

MCP Apps is a standard extension to the Model Context Protocol (MCP) that lets an MCP server return an interactive HTML UI for a tool result. The UI renders inline in Trimble Assist and communicates with the MCP Apps host using JSON-RPC over `postMessage`.

Trimble Assist supports standard MCP Apps. We also provide an optional Trimble extension for one extra use case: letting an MCP App call tools registered by the embedding host application. That host-proxy behavior is **not** standard MCP Apps; it is our custom SDK/protocol that runs alongside the standard MCP Apps protocol.



FOR TRIMBLE ASSIST USERS

If you are embedding Trimble Assist with the Agentic Platform Iframe SDK or framework components, you do not need to implement the MCP Apps host-side bridge yourself. Trimble Assist handles sandboxing, nested iframes, message routing, and standard MCP Apps protocol support.

What Runs Where

There are four actors in the embedded setup:

1. **Host Application:** The application embedding Trimble Assist. It is not part of standard MCP Apps, but it can optionally expose host-local tools through our Trimble extension.
2. **Trimble Assist Chat UI:** The embedded chat iframe. It renders the conversation, hosts MCP App iframes, and bridges protocol messages.
3. **Agent Service and MCP Server:** The remote service path that runs standard MCP tools and returns MCP App UI resources.
4. **MCP App:** Your interactive HTML UI rendered inside the chat as a nested iframe.

The important distinction:

- Standard MCP Apps talk to their MCP server through the MCP Apps SDK.
- The Trimble host-proxy extension lets an MCP App talk to the embedding host application through our custom SDK.

Standard MCP Apps Flow

This is the normal MCP Apps flow. The MCP server owns the tool and the UI resource. Trimble Assist acts as the MCP Apps host and bridges messages between the nested app iframe and the remote MCP server path.



Quick Start

To build an MCP App, you first need to install the SDK:

```
npm install @modelcontextprotocol/ext-apps
```

Then, initialize the `App` and connect it to the parent window (the Chat UI iframe) using the `PostMessageTransport`:

```
import { App, PostMessageTransport } from '@modelcontextprotocol/ext-apps';
```

```
const app = new App({
  name: 'My MCP App',
  version: '1.0.0',
});

// Connect to the parent Chat UI iframe
const transport = new PostMessageTransport(window.parent);
await app.connect(transport);
```

Receiving Tool Data

Once connected, your app needs to listen for data from the MCP Apps host. In Trimble Assist, the chat UI is the host for the standard MCP Apps protocol. This happens through three main lifecycle events:

```
// 1. Tool Input: Called when the tool starts executing.
// Good for showing loading states or initializing UI with parameters.
app.ontoolinput = ({ arguments: toolArgs }) => {
  console.log('Tool is running with:', toolArgs);
  showLoadingSpinner();
};

// 2. Tool Result: Called when execution completes successfully.
// This is where you receive the data to render your UI.
app.ontoolresult = ({ content, structuredContent }) => {
  hideLoadingSpinner();
  // structuredContent is optimized for UI rendering (JSON data)
  console.log('Data received:', structuredContent);
  renderDashboard(structuredContent);
};

// 3. Tool Cancelled: Called if the user or host cancels the execution.
app.ontoolcancelled = ({ reason }) => {
  hideLoadingSpinner();
  showErrorMessage(`Cancelled: ${reason}`);
};
```

Standard MCP Apps Actions

When a user interacts with your UI (for example, clicking a button or submitting a form), you can send actions through the standard MCP Apps SDK. These methods either communicate with the originating MCP server or update the chat context.

Fetching More Data (`callServerTool`)

Run another tool on the remote MCP server in the background to fetch more data without adding a new message to the chat. This request is proxied through the Agent Service to the remote MCP server.

```
async function refreshData(timeRange: string) {
  const result = await app.callServerTool({
    name: 'get_metrics',
    arguments: { period: timeRange },
  });

  if (!result.isError && result.structuredContent) {
    updateDashboard(result.structuredContent);
  }
}
```

Saving State/Memory (`updateModelContext`)

Save the user's selections so the AI remembers them for the next chat message. **This does not trigger a chat response.** It silently updates the AI's context.

```
async function saveUserSelection(metricId: string) {
  await app.updateModelContext({
    content: [
      {
        type: 'text',
        text: `The user is currently looking at metric: ${metricId}`,
      },
    ],
  });
}
```

Triggering a Chat Response (`sendMessage`)

Force the chat to send a new message on behalf of the user, prompting the AI to respond.

```
async function askAiToAnalyze(data: unknown) {
  await app.sendMessage({
    role: 'user',
    content: [
      {
        type: 'text',
        text: `Please analyze this specific data point:
${JSON.stringify(data)}`,
      },
    ],
  });
}
```

Trimble Extension: Host-Proxy Tools

Standard MCP Apps do not define a way for an app UI to call tools registered by the application embedding Trimble Assist. That WebMCP-like host-proxy behavior is the Trimble extension.

The extension uses a dedicated `postMessage` channel alongside the standard MCP Apps bridge. It lets an MCP App discover and call tools that the embedding host application explicitly exposed to apps. This bypasses the remote MCP server.

Executing Local Tools (`callLocalTool`)

Run a tool directly in the host application. To use this, import `callLocalTool` from the Trimble MCP app tools package.

```
npm install @trimble-agentic/agentic-mcp-app-tools
```

```
import {
  callLocalTool,
  LocalToolError,
} from '@trimble-agentic/agentic-mcp-app-tools';

async function triggerLocalAction(data: string) {
  try {
    const result = await callLocalTool('save_file', {
      path: '/tmp/out.txt',
      content: data,
    });
  } catch (error) {
    console.error('Error calling local tool:', error);
  }
}
```

```

    });

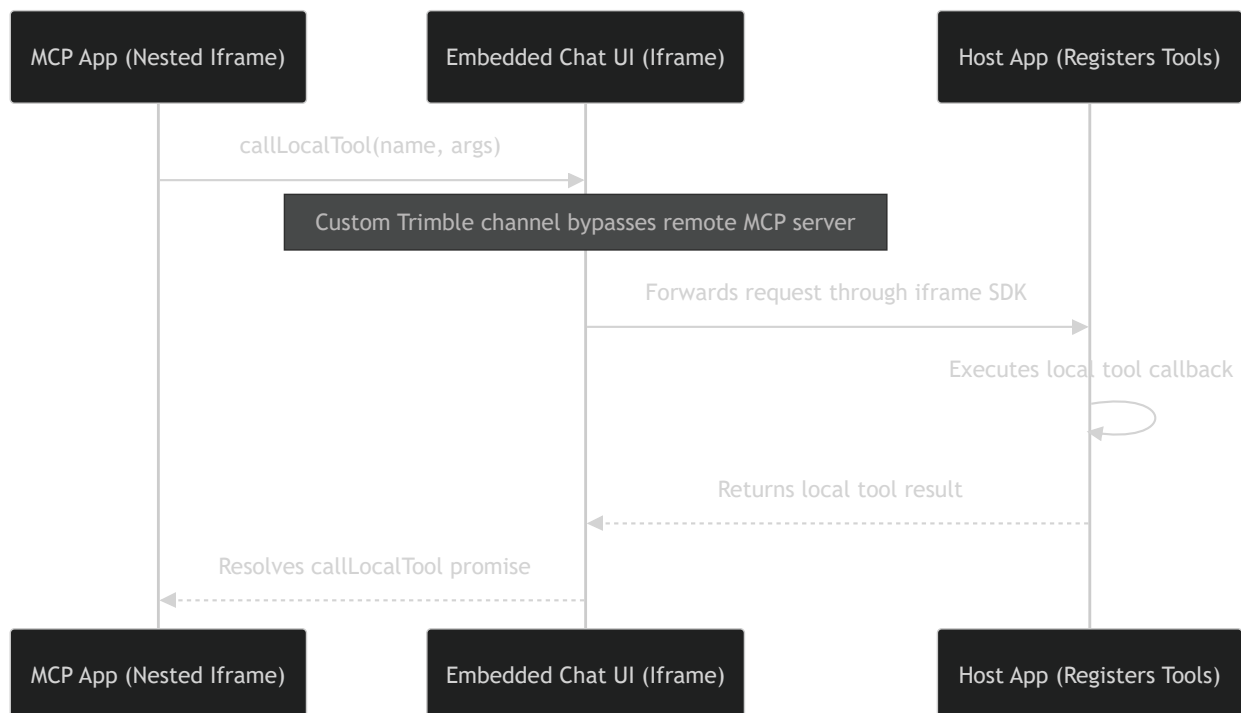
    if (result.isError) {
        console.warn('Local tool returned an error:', result.content);
        return;
    }

    console.log('Local tool executed successfully:', result.content);
} catch (error) {
    if (error instanceof LocalToolError) {
        console.error('Local tool could not run:', error.code,
error.message);
        return;
    }

    console.error('Local tool failed:', error);
}
}

```

Local Tool Execution Flow



Discovering Host Tools

The same package also exposes read-only discovery helpers. Use them when your app should adapt to the host application's capabilities.

One-Shot Read

```
import { getHostTools } from '@trimble-agentic/agentic-mcp-app-tools';

const hostTools = await getHostTools();
const canSaveFiles = hostTools.some(tool => tool.name === 'save_file');
```

Live Subscription

Use `subscribeHostTools()` to receive the initial catalogue and every subsequent change. The listener fires with the current set each time it updates.

```
import {
  subscribeHostTools,
  type HostTool,
} from '@trimble-agentic/agentic-mcp-app-tools';

const unsubscribe = subscribeHostTools((tools: HostTool[]) => {
  const canSaveFiles = tools.some(tool => tool.name === 'save_file');
  toggleSaveButton(canSaveFiles);
});

// Later, when the component unmounts:
unsubscribe();
```

Each `HostTool` is a structural subset of the AG UI tool definition:

```
type HostTool = {
  name: string;
  description: string;
  parameters?: unknown; // The tool's JSON schema, parsed by the app as
  needed.
};
```

Further Reading

- [MCP Apps Specification](#)
- [MCP Apps SDK on npm](#)
- [Model Context Protocol](#)
- [Iframe integration](#)

